

LAB 4 — TREASURE HUNT

Code Changes Documentation

Task	Implemented feature
Task 1	Floor traps — reset player to start
Task 2	Enemy guard — patrol and reset on collision
Task 3	Mini-map — walls, floor, player position
Task 4	Inventory UI — key slot and key icon

This document records the changes made to the existing Treasure Hunt Pygame project. The original procedural dungeon generation, player movement, key/chest mechanics and restart flow were retained.

1. Existing Project Structure

The original game uses a 20×15 grid. Rooms are generated procedurally, corridors connect rooms, the first room is the player start, and the final rooms contain the chest and key. Movement checks the destination rectangle against WALL cells.

```
TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY = 0, 1, 2, 3
SPEED = 3
```

The implementation was extended in place rather than replacing the game architecture.

2. Task 1 — Floor Traps

Requirement: traps send the player back to the starting position and display a clear status message.

2.1 New trap tile

```
WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4
SPEED = 3
TRAP_COUNT = 4
```

2.2 Trap placement

```
if rooms:
    start_room = rooms[0]
    trap_cells = []

    for r in range(ROWS):
        for c in range(COLS):
            if grid[r][c] == FLOOR and not start_room.collidepoint(c, r):
                trap_cells.append((r, c))

    trap_count = min(TRAP_COUNT, len(trap_cells))

    for _ in range(trap_count):
        r, c = random.choice(trap_cells)
        grid[r][c] = TRAP
        trap_cells.remove((r, c))
```

Traps are placed only on walkable floor cells outside the starting room.

2.3 Reset and status message

```
self.start_pos = (sx, sy)

if cell == TRAP:
    self.player.rect.topleft = self.start_pos
    self.status = "Trap! Back to start!"
```

2.4 Trap rendering

```
elif cell == TRAP:
    pygame.draw.line(
        self.screen, (90, 30, 30),
        (c*TILE+8, r*TILE+8),
        (c*TILE+TILE-8, r*TILE+TILE-8), 4
    )
    pygame.draw.line(
        self.screen, (90, 30, 30),
        (c*TILE+TILE-8, r*TILE+8),
        (c*TILE+8, r*TILE+TILE-8), 4
    )
```

3. Task 2 — Enemy Guard

Requirement: a guard patrols back and forth near the chest. Touching it resets the player to start.

3.1 Guard definition

```
GUARD_SIZE = 26
GUARD_SPEED = 2

class Guard:
    def __init__(self, point_a, point_b):
        self.rect = pygame.Rect(point_a[0], point_a[1],
                                GUARD_SIZE, GUARD_SIZE)
        self.point_a = pygame.Vector2(point_a[0], point_a[1])
        self.point_b = pygame.Vector2(point_b[0], point_b[1])
        self.target = self.point_b
        self.speed = GUARD_SPEED
        self.color = (200, 60, 60)
```

3.2 Patrol movement

```
def update(self):
    current = pygame.Vector2(self.rect.x, self.rect.y)
    direction = self.target - current

    if direction.length() <= self.speed:
        self.rect.topleft = (int(self.target.x), int(self.target.y))
        if self.target == self.point_b:
            self.target = self.point_a
        else:
            self.target = self.point_b
    else:
        direction = direction.normalize()
        current += direction * self.speed
        self.rect.topleft = (int(current.x), int(current.y))
```

3.3 Placement near chest

```
if chest_room:
    guard_y = chest_room.centery * TILE + 7
    left_x = chest_room.left * TILE + 7
    right_x = (chest_room.right - 1) * TILE + 7

    self.guard = Guard((left_x, guard_y), (right_x, guard_y))
```

3.4 Collision behavior

```
if self.guard:
    self.guard.update()

if self.player.rect.colliderect(self.guard.rect):
    self.player.rect.topleft = self.start_pos
    self.status = "Guard caught you! Back to start!"
```

4. Task 3 — Mini-map

Requirement: a small real-time mini-map shows the dungeon layout, distinguishes walls from floor, and marks the player.

4.1 Mini-map setup

```
map_width = 180
map_height = 135
map_x = WIDTH - map_width - 10
map_y = 10
```

```
cell_w = map_width / COLS
cell_h = map_height / ROWS
```

4.2 Draw walls and floor

```
for r in range(ROWS):
    for c in range(COLS):
        cell = self.grid[r][c]

        if cell == WALL:
            color = (35, 35, 45)
        else:
            color = (190, 190, 175)

        pygame.draw.rect(
            self.screen, color,
            (map_x + c*cell_w, map_y + r*cell_h,
             cell_w + 1, cell_h + 1)
        )
```

4.3 Mark the player

```
player_col = self.player.rect.centerx // TILE
player_row = self.player.rect.centery // TILE

pygame.draw.rect(
    self.screen, (60, 120, 220),
    (map_x + player_col*cell_w,
     map_y + player_row*cell_h,
     cell_w + 1, cell_h + 1)
)
```

The mini-map is redrawn every frame, so the player marker follows movement.

5. Task 4 — Inventory UI

Requirement: an inventory slot is empty initially and displays a key icon immediately after key collection.

5.1 Existing key state

```
self.has_key = False
```

The existing `Player.has_key` state is reused, avoiding a separate inventory dependency.

5.2 Inventory slot

```
slot_size = 45
slot_x = WIDTH - 60
slot_y = ROWS * TILE + 12

pygame.draw.rect(
    self.screen, (60, 60, 75),
    (slot_x, slot_y, slot_size, slot_size),
    border_radius=5
)
pygame.draw.rect(
    self.screen, (130, 130, 140),
    (slot_x, slot_y, slot_size, slot_size),
    2, border_radius=5
)
```

5.3 Key icon when collected

```
if self.player.has_key:
    center_x = slot_x + slot_size // 2
    center_y = slot_y + slot_size // 2

    pygame.draw.circle(
        self.screen, (255, 230, 50),
        (center_x - 7, center_y - 4), 7, 3
    )

    pygame.draw.line(
        self.screen, (255, 230, 50),
        (center_x, center_y - 4),
        (center_x + 13, center_y - 4), 4
    )
```

Because the slot is rendered from `self.player.has_key` each frame, the UI updates immediately after pickup.

6. Final Testing Checklist

Feature	Expected result
Trap	Player returns to start; status shows trap message.
Guard	Guard patrols between two points; collision returns player to start.
Mini-map	Walls/floor are distinguishable and player marker updates.
Inventory	Empty before key; key icon appears after pickup.
Key + Chest	Existing treasure-found flow still works.
Restart	R regenerates/resets the run.

7. Summary

- Added floor traps and reset/status behavior.
- Added a patrolling guard and collision reset behavior.
- Added a real-time dungeon mini-map with a player marker.
- Added an inventory slot with a key icon after pickup.
- Retained procedural generation, movement/collision, key/chest mechanics, and restart functionality.
- No additional external dependency was introduced; the features use the existing Pygame setup.

Prepared as the code-change documentation for the Lab 4 submission.